

Tel-Aviv University

LAB B

FPGA Experiment: Stopwatch

Student 1

Name: *Saleh Khalil*

ID: *213310485*

Student 2

Name: *Noor Mahameed*

ID: *213534613*

Submission Date: 14 January 2026

1 Part 1: Full Adder

Question

Implement $FA(a,b,ci,sum,co)$ such that $a + b + ci = 2 \cdot co + sum$.

Answer

(b) Truth table:

a	b	ci	sum	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

We have carry out when at least two of $\{a,b,ci\}$ are 1.

(c) Schematic using AND/OR/XOR only

$$sum = a \oplus b \oplus ci \quad co = (a \wedge b) \vee (a \wedge ci) \vee (b \wedge ci)$$

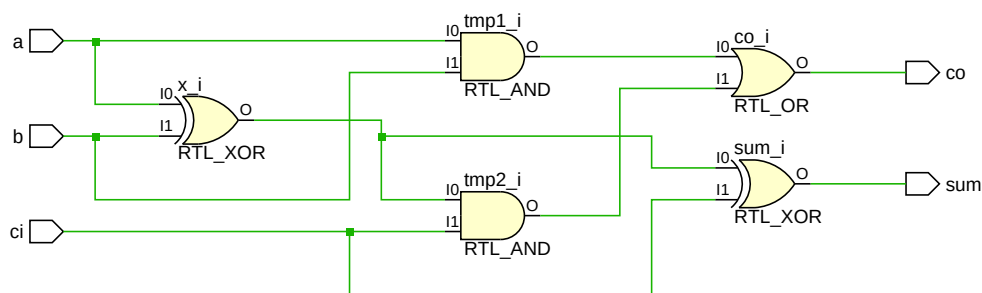


Figure 1: (Part 1) Full Adder schematic (AND/OR/XOR only).

(f) Annotated waveform

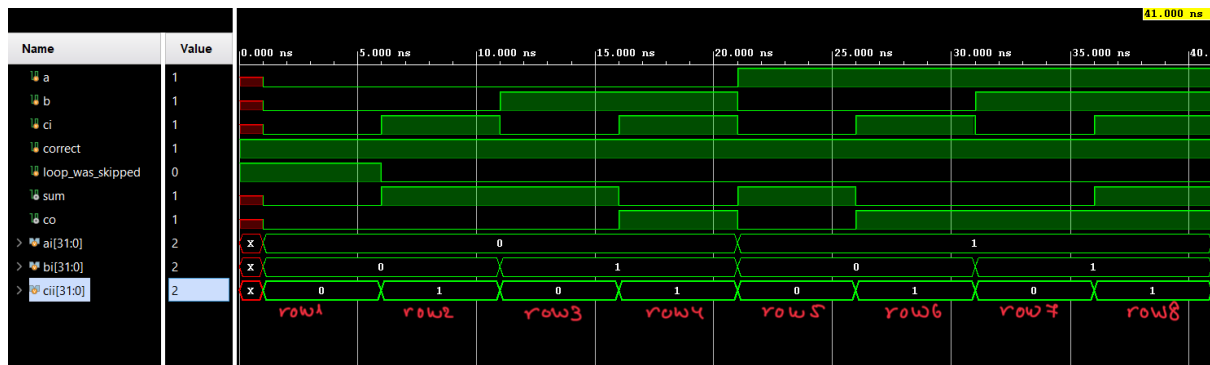


Figure 2: (Part 1) FA simulation waveform: As we can see the simulation matched our truth table.

2 Part 2: Conditional Sum Adder

Question

Implement $\text{CSA}(a, b, ci, \text{sum}, co)$ with parameter N such that:

$$\langle \text{sum} \rangle + 2^N \cdot co = \langle a \rangle + \langle b \rangle + ci$$

Must use FA + **generate** + recursive construction.

Answer

(a) Block diagram

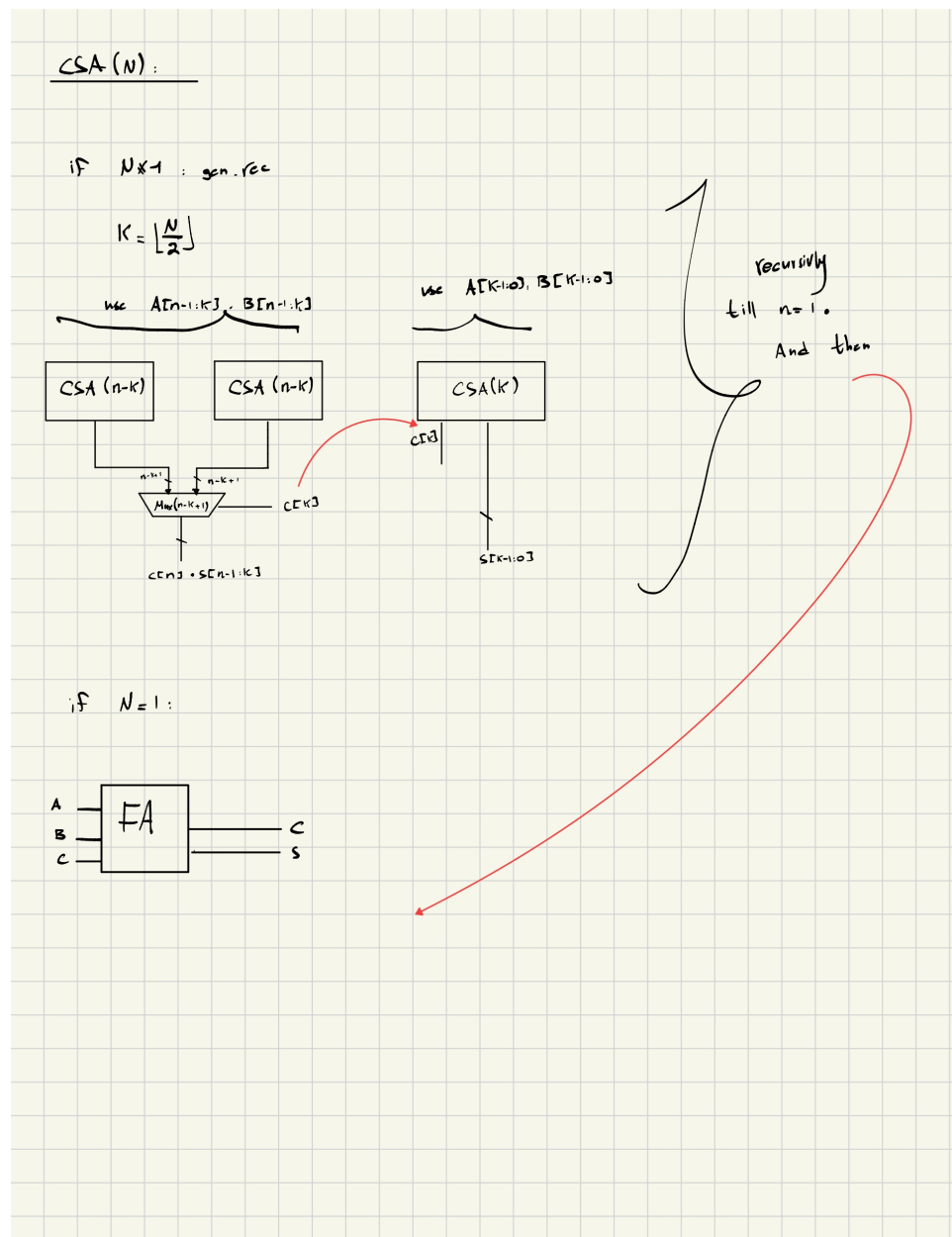


Figure 3: (Part 2) CSA(N) block diagram.

(c) Timing improvement vs ripple-carry; when ripple preferred

Conditional Sum Adder improves timing by precomputing results for both $C_{in} = 0$ and $C_{in} = 1$ in parallel, then selecting the correct one using a multiplexer tree. This reduces the critical path compared to a ripple-carry adder, whose delay grows roughly linearly with the number of bits. A ripple-carry adder is still better for small bit-widths paths because it is smaller and simpler.

(d) What is a generate block? Why needed here?

Recursion describes topology (the connection of different wires). Since we can't generate UUTs inside always blocks(it will cause compilation errors - check # for reason), we'll need a generate block that will predefine (in elaboration - it will resolve the code into a flat physical netlist) our topology and connectivity(predefine as in pre-simulation). Without a generate block, the compiler wouldn't know how many FAs/MUXs to create before simulation, which is impossible since modules are static (hardware).

If we instantiate a module inside its code without the use of a generate block, the compiler will notice an infinite loop that will raise a fatal error and will stop compiling the code.

(g) Annotated waveform proving conditional behavior



Figure 4: CSA(3) waveform

(h) Repeat for CSA(4)

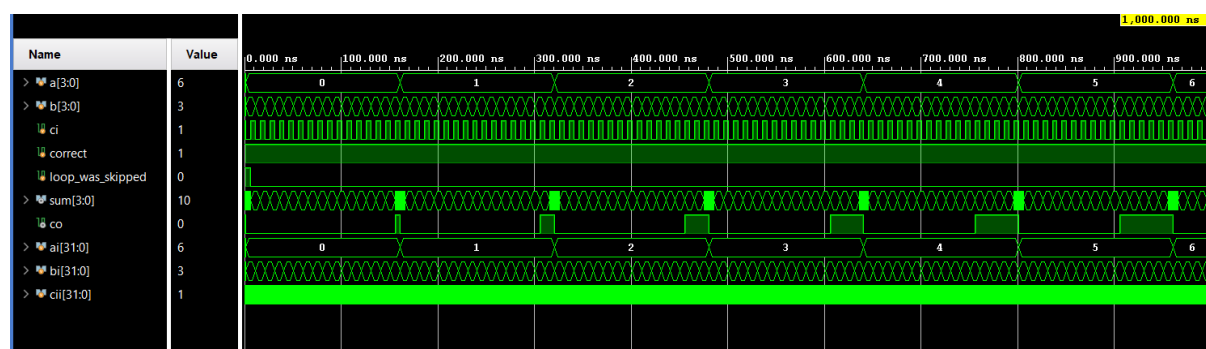


Figure 5: CSA(4) waveform

3 Part 3: Limited Incrementor

Question

Implement $\text{Lim_Inc}(a, ci, sum, co)$ with parameter L and $N = \lceil \log_2 L \rceil$:

$$\langle a \rangle + ci \geq L \iff (co = 1 \wedge \langle sum \rangle = 0), \quad \langle a \rangle + ci < L \iff (co = 0 \wedge \langle sum \rangle = \langle a \rangle + ci)$$

Answer

(a) Block diagram

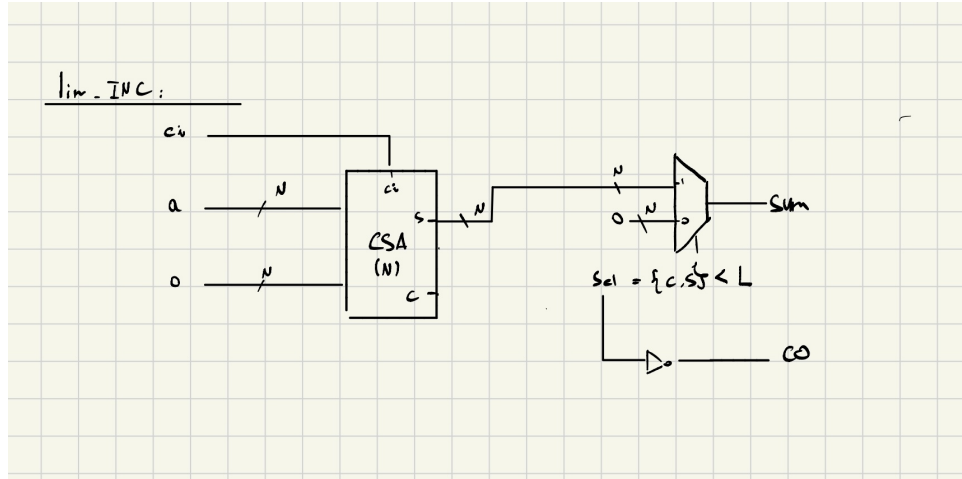


Figure 6: (Part 3) $\text{Lim_Inc}(L)$ block diagram.

(e) Why raw sum differs when never differs simpler design

sum differs from the raw adder result only when the limit is reached $a + ci \geq L$ the module forces $sum=0$ and sets $co=1$, instead of outputting $a + ci$.

They never differ when $L = 2^N$ (or larger), because the only wrap case is $2^N - 1 + 1 \rightarrow 0$ anyway. In that case a normal N -bit adder is enough: $sum = a + ci$ and $co = \text{carry-out}$.

(f) Annotated waveform if the simulation

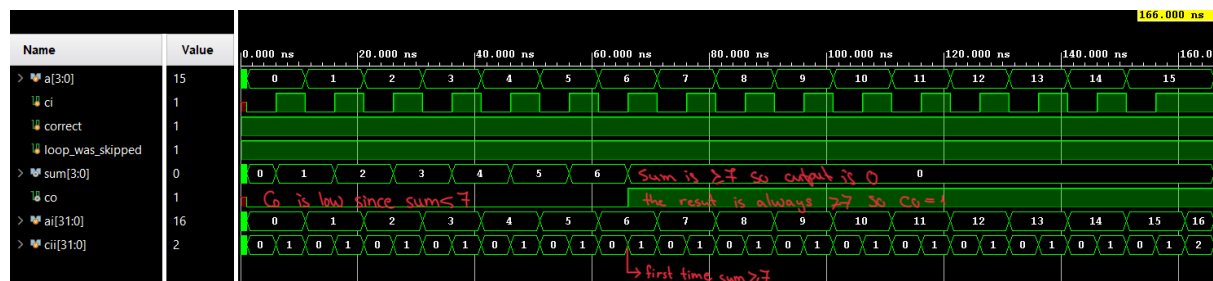


Figure 7: (Part 3) Lim_Inc waveform: as long as a is smaller than 6 the output is $sum = a + ci, co = 0$ after $a = 6$ and $ci = 1$ the output is constant with $sum = 0, co = 1$.

4 Part 4: Counter

Question

Implement `Counter(clk, init_regs, count_enabled, time_reading)` with parameter `CLK_FREQ`. `time_reading[7:0]` is $\{dasec, sec\}$ (decimal digits) representing seconds modulo 100.

Answer

(a) Range

`time_reading` can represent numbers from 00 to 99 seconds.

(b) Use `Lim_Inc` + register for 1Hz tick

Using a limited incrementer with $L = 10^8$ provides a threshold of 100M ticks its output is stored in a register and fed back to the incrementer input. Thus a increments each cycle until $a = L$, at which point the carry-out asserts to indicate that one second has passed.

(c) What determines number of `Lim_Inc` modules?

One for clock divider + one per decimal digit counter (sec ones and sec tens) $\Rightarrow$ typically 3.

(d) Choose `L` for 0.1Hz tick

Either hierarchical decimal carry (tens-of-seconds increments every 10 seconds) or direct divider with $L = 10 \cdot CLK_FREQ$.

(e) If clock is 50MHz

If the clock frequency changed, we simply just change `CLK_FREQ` to 50MHz (`L` of the first `lim_inc`) and we don't change anything else in the design, since by doing this we'll adjust the first incrementor to count every 50M clock cycles and its co will be 1hz as wanted therefore the second `lim_inc` co will be 0.1hz also as wanted!

(f) Block diagram

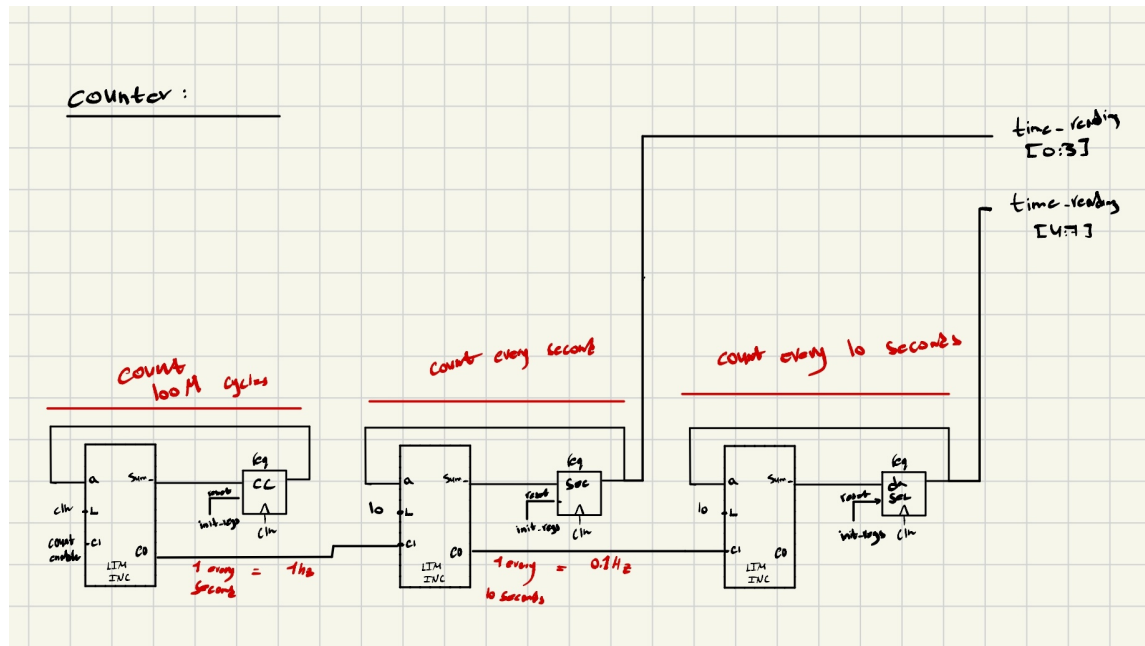


Figure 8: (Part 4) Counter block diagram.

(j) Annotated waveform



Figure 9: (Part 4) Counter waveform.

(k) Why Lim_Inc cannot hold count internally

No clock input: counting requires sequential state (registers), so state must live outside Lim_Inc.

(l) Maximum time before roll-over

99 seconds then back over to 00, since we only have 2 digits and count in seconds only.

(m) Extend to MM:SS

We will always need one `lim_inc` instance to count clock cycles, denoted by $cc(t)$. The total number of `lim_inc` instances is determined by how many seconds we want to count before rolling over. For example, to count up to 1000 seconds we need 3 `lim_inc` blocks, and to count up to 10000 seconds we need 4 `lim_inc` blocks. In general,

$$N_{\text{lim_inc}} = 1 + \lfloor \log_{10}(\text{Max_seconds}) \rfloor.$$

5 Part 5: Stash

Question

Store the last DEPTH samples (8-bit). Overwrite oldest if full. next_sample cycles displayed sample. New write must appear immediately on sample_out.

Answer

(a) Block diagram

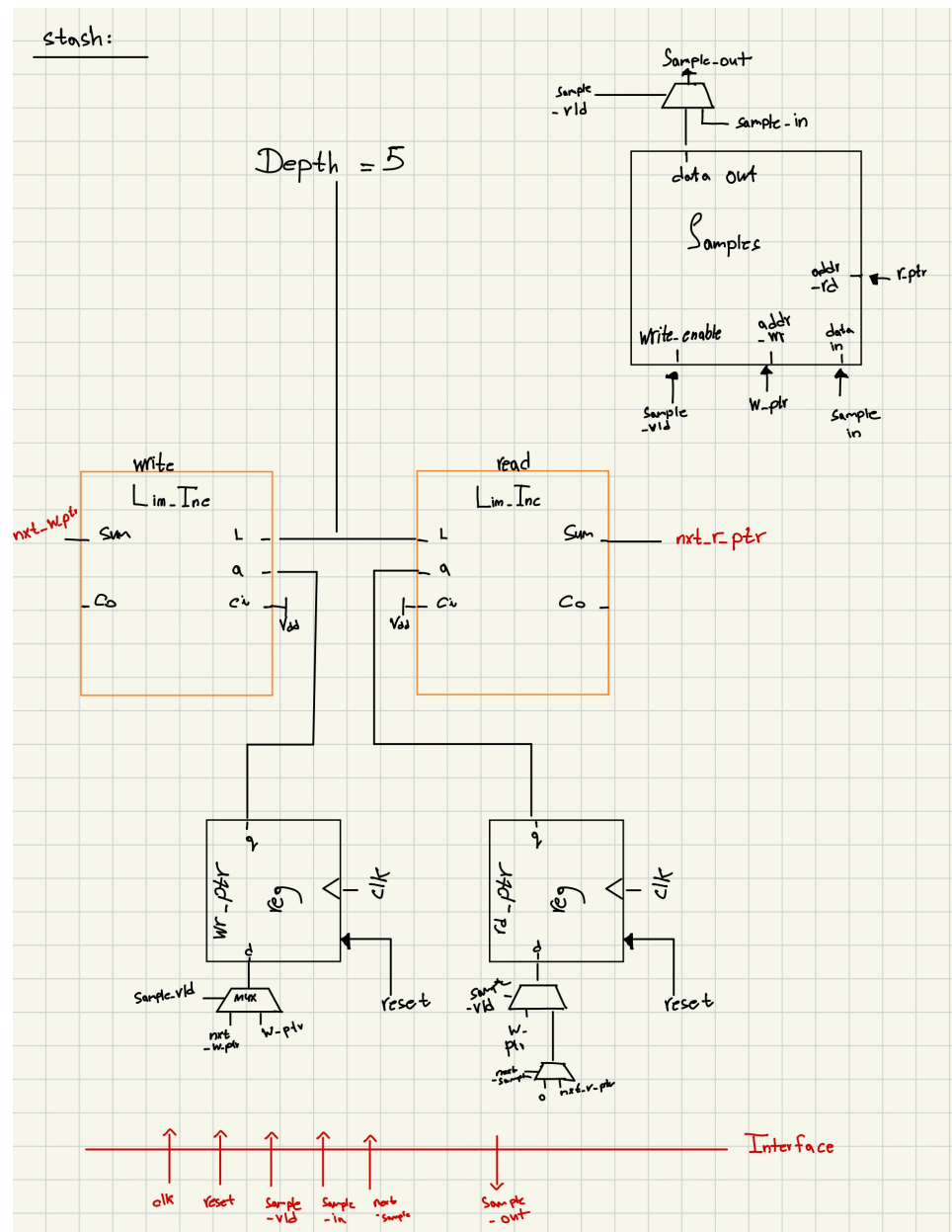


Figure 10: (Part 5) Stash block diagram.

(e) Waveform: immediate update on write

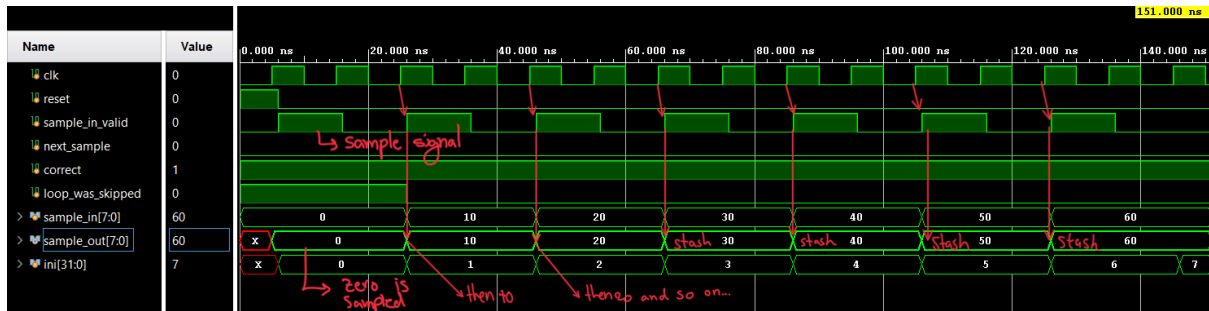


Figure 11: (Part 5) Stash waveform: We can immediately see the change in sample out since `sample_out` is assigned directly to `sample_in` on `sample_in_valid` which eliminate any delays.

(f) Explain selection logic

```
assign sample_out = (sample_in_valid)? sample_in : samples[r_ptr];
```

when valid is high the output is the input giving on cycle for the *sample* to store the value at read pointer (`r_ptr`) to then read as the output.

(g) Alternative approach + pros/cons

Alternative: Use a FIFO with a `head` register that always holds the next output sample (prefetch from memory, and load `head` when empty on a write).

Pros: `sample_out` is stable (no glitches) and easier to meet timing.

Cons: More complex, and the output changes only on the next clock so `sample_in` must be synchronized with the clock.

6 Part 6: Control

Question

Implement Ctl according to the provided Mealy FSM diagram.

Answer

(a) Moore or not?

Although the state diagram is drawn in a Mealy-like style, the controller is in fact a **Moore** finite state machine, because its outputs depend only on the current state and are independent of the input signals. The input signals affect only the next-state transitions, not the output values themselves.

(b) Block diagram

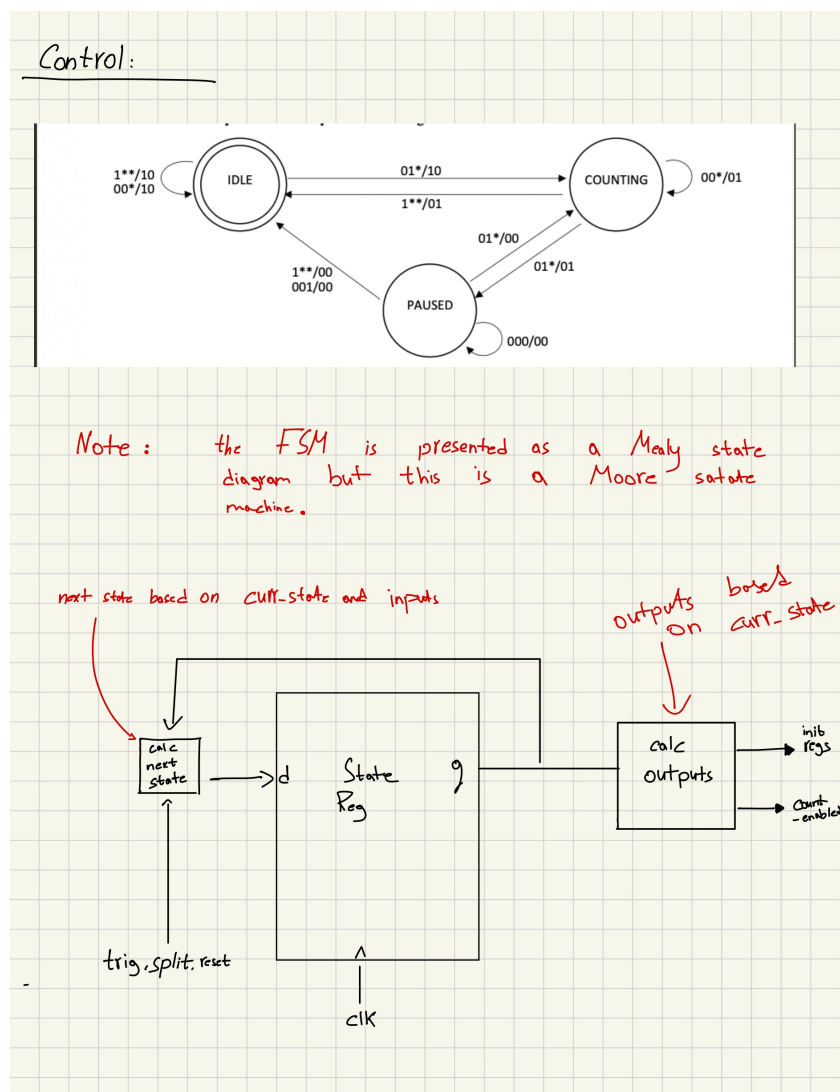


Figure 12: (Part 6) Control module FSM diagram.

(f) Waveform

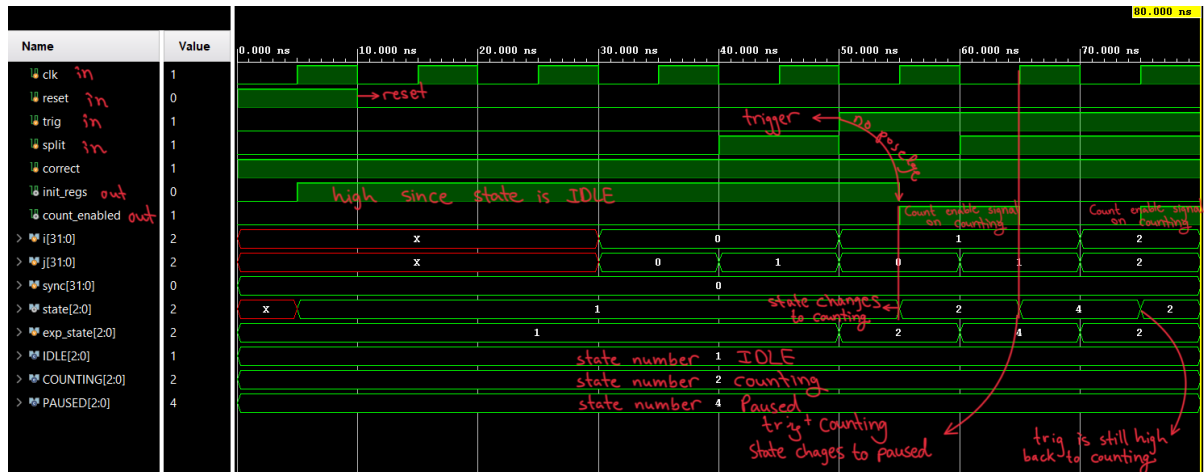


Figure 13: (Part 6) Control waveform.

7 Part 7: Debouncer

Question

Implement Debouncer using a saturated up/down counter. Output must be a single-cycle pulse when a stable press is detected.

Answer

(a) Block diagram

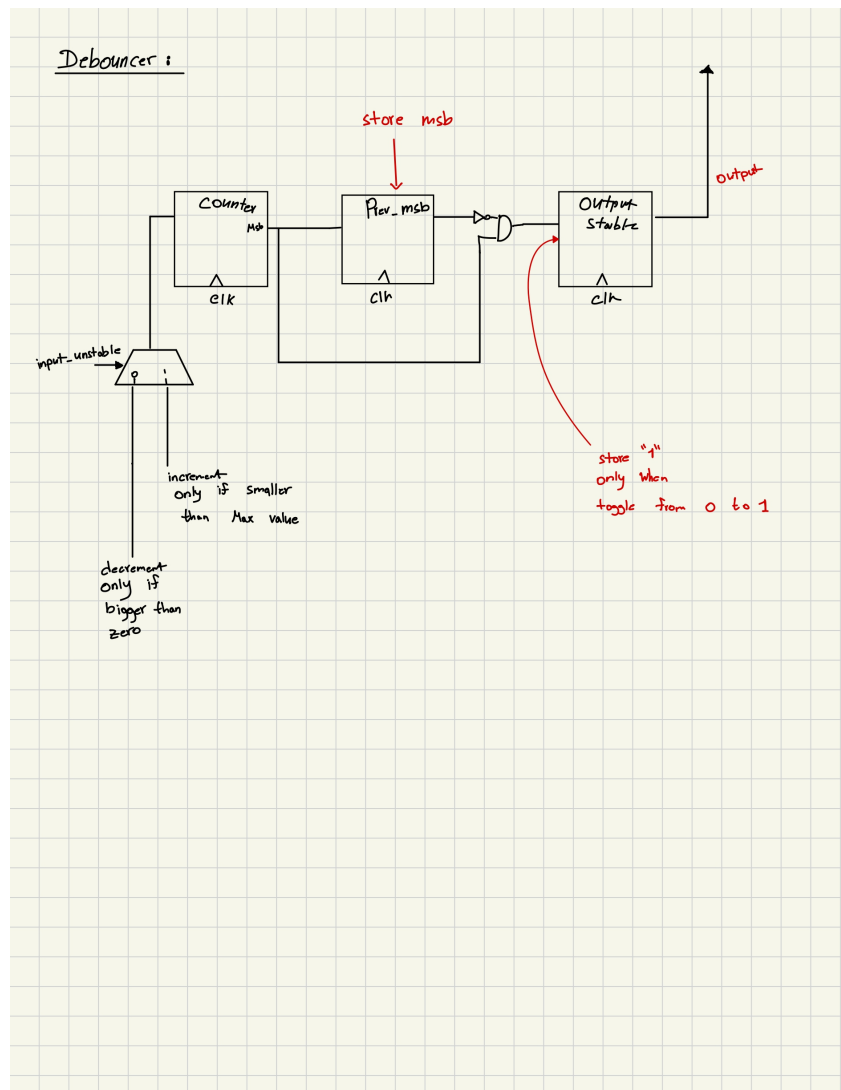


Figure 14: (Part 7) Debouncer block diagram.

(d) Tradeoff between choosing high and low values for the counter

small counter → faster response but more false triggers from bounce/noise.

large counter → less noise but more delay and can miss short presses.

This matches the low-pass filter behavior because it cancels fast changing signals (high-frequency/ bouncing).

8 Part 8: Seven Segment Display

Question

Choose `clkdiv` bits to scan 4 digits such that full refresh time is 1–16ms. Justify analytically.

Answer

(b) Choose `clkdiv` bits

```
assign s = clkdiv[19:18];
```

(c) Justification

If digit advance occurs every 2^k clocks:

$$T_{\text{digit}} = \frac{2^k}{f_{\text{clk}}}, \quad T_{\text{display}} = \frac{4 \cdot 2^k}{f_{\text{clk}}}$$

With `s = clkdiv[19:18]`, the LSB of `s` is bit 18, so $k = 18$ and $f_{\text{clk}} = 100 \text{ MHz} = 10^8 \text{ Hz}$.

Time per digit:

$$T_{\text{digit}} = \frac{2^{18}}{10^8} = \frac{262,144}{10^8} = 2.62 \text{ ms}$$

Full display time:

$$T_{\text{display}} = \frac{4 \cdot 2^{18}}{10^8} = 4 \times 2.62 \text{ ms} = 10.49 \text{ ms}$$

$$1 \text{ ms} \leq 10.49 \text{ ms} \leq 16 \text{ ms} \quad \checkmark$$

Therefore, using bits `[19:18]` is a good choice.

9 Part 9: Stopwatch

Question

Integrate all modules into `Stopwatch.v` with debounced buttons, selection toggle, stash sampling, split behavior, and 7-seg output. Provide elaborated and synthesized schematics, and fix critical warnings.

Answer

(a) Block diagram

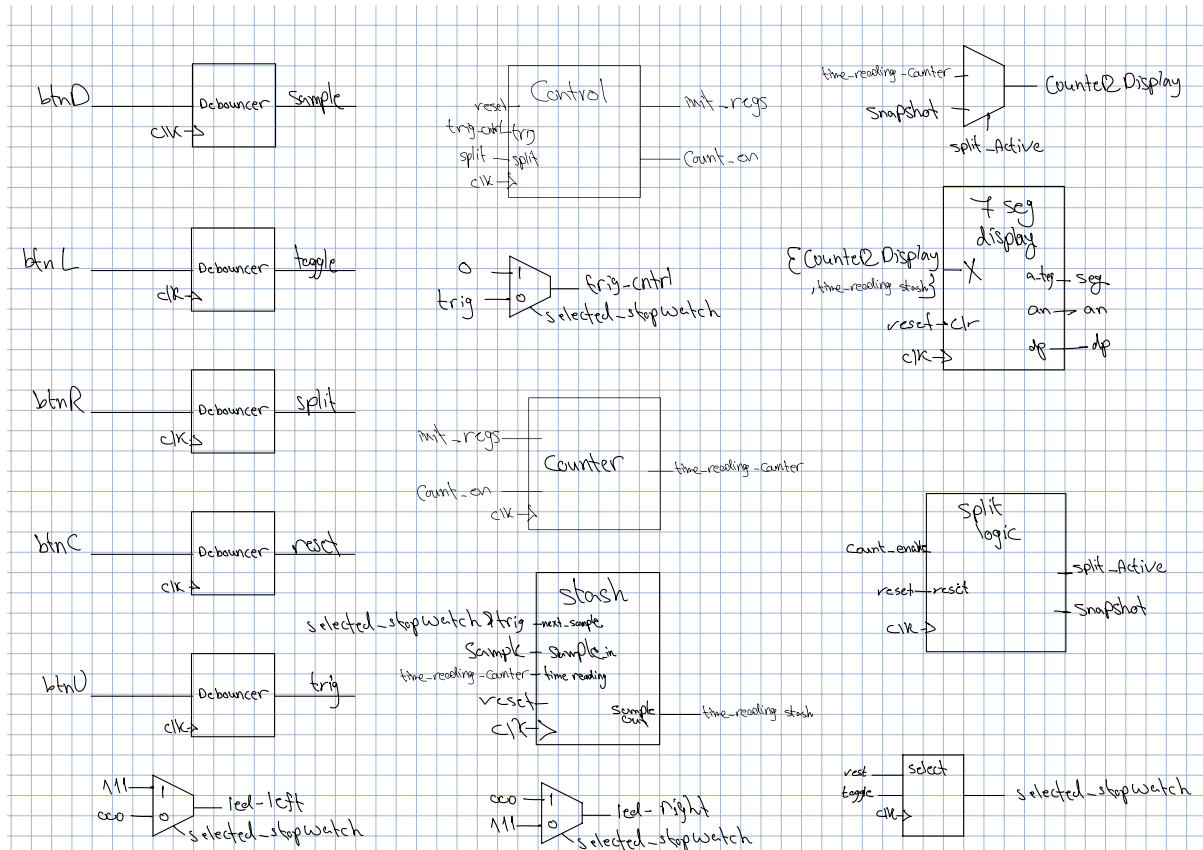


Figure 15: (Part 9) Stopwatch top-level block diagram.

Each input for the board button must go thru a *Debouncer* first. and the signal then the is passed to the corresponding blocks except of trigger since it can serve as next_sample too so we pass it thru a Mux with selected_stopwatch as Mux control.

Elaborated Design Schematic

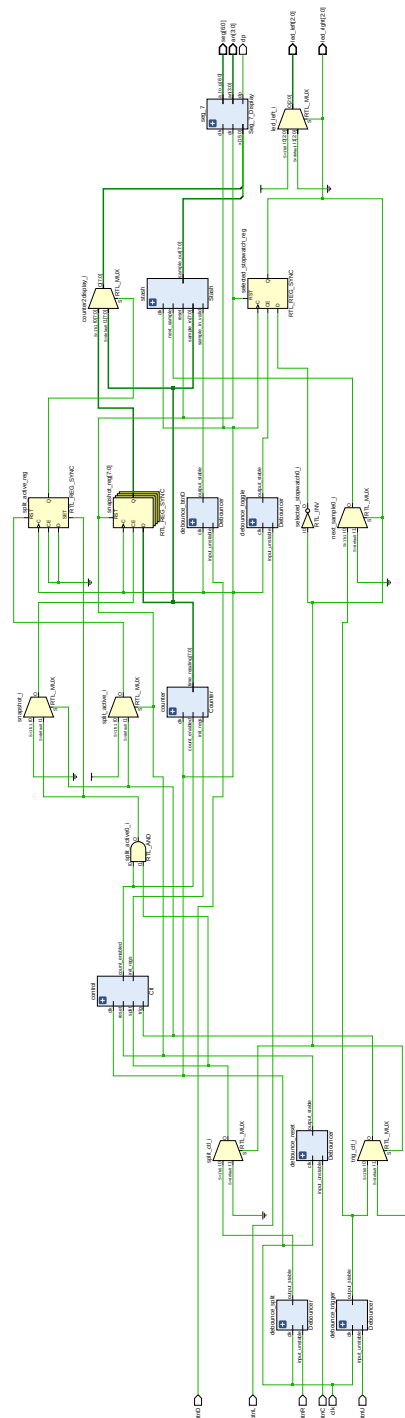


Figure 16: (Part 9) Stopwatch top-level block diagram.

9.1

Differences: The elaborated view shows the RTL structure exactly as written in Verilog, before any optimization (modules, registers, and logic connections). The synthesized schematic shows the optimized implementation and the actual FPGA resources used to realize that Verilog design.

10 Part 10: Timing Analysis

Timing summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.652 ns	Worst Hold Slack (WHS): 0.241 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 494	Total Number of Endpoints: 494	Total Number of Endpoints: 232
All user specified timing constraints are met.		

Figure 17: (Part 10) Timing summary

As shown in the timing summary, all user-specified timing constraints are met. To estimate the critical path delay we use:

$$longest_path = clock_period - setup_min_slack. \quad (1)$$

With a 100 MHz clock, $clock_period = 10$ ns, and from the report $WNS = 2.652$ ns, hence $longest_path = 10 - 2.652 = 7.348$ ns; additionally, hold and pulse-width checks pass with $WHS = 0.241$ ns and $WPWS = 4.500$ ns.